

Salesforce go-live runbook

The Case → plan → FreeWheel path is built and unit-tested; going live is a configuration + credentials exercise, not new code. This is the checklist.

1. Build the Case fields (admin)

Give the Salesforce admin `docs/salesforce-case-fields.csv` — it is generated from the live config (`python scripts/build_salesforce_fields.py`) and lists every field the automation reads: API name, type, picklist values (all 15 regions, every campaign, all brands/formats), and help text. Add the Status/Reason picklist values at the bottom. Regenerate + re-share whenever brands/regions change.

The API names in the CSV are what the code reads (`CASE_FIELD_MAP` in `integrations/salesforce.py`). If the org must use different API names, edit `CASE_FIELD_MAP` (and `CASE_FIELD_SPEC`) to match — one place, and the CSV + parser stay in sync.

2. Credentials (.env)

```
SALESFORCE_USERNAME=...
SALESFORCE_PASSWORD=...
SALESFORCE_SECURITY_TOKEN=...      # omit if org has IP relaxation
SALESFORCE_DOMAIN=test             # 'test' for sandbox, 'login' for production
# optional Connected App:
SALESFORCE_CLIENT_ID=...  SALESFORCE_CLIENT_SECRET=...  SALESFORCE_INSTANCE_URL=...
```

Install the extra: `pip install -e '.[salesforce]'` .

3. Preflight (proves creds + schema)

```
promo-ops salesforce-check
```

Logs in and describes the Case object, then reports any **missing fields** or **missing Status/Reason picklist values** vs. what the automation needs. Green = the org is ready. (Pure check is unit-tested via `check_case_schema .`)

4. Attach the Targeting sheet

Detailed targeting (showlist / genres / networks / Pluto channels+categories) is the standard **Targeting** sheet attached to the Case; the automation downloads the latest attachment whose title contains "Targeting" and parses it with the same parser the sheet/YAML paths use. For Pluto, use the region's real names — see `docs/PLUTO_TARGETING_NAMES.md`.

5. Run a Case end-to-end

```
promo-ops from-case <CASE_ID>                # validate + build + dry-run (review, no writes)
promo-ops from-case <CASE_ID> --live           # create the NOT_BOOKED draft in FreeWheel
promo-ops poll-cases [--live]                  # process every Case flagged "Ready for Automation"
```

The flow: read Case core fields (`build_plan_dict`) + attached Targeting → `support_plan_from_dict` → `OrderBuilder` → FreeWheel draft. On success the automation sets Reason = "Submitted to FreeWheel" and comments the IO link; if it can't build, it sets Status = "Needs Info" with the reason.

6. Schedule the poller (unattended)

`poll-cases` processes every Case flagged "Ready for Automation". It is **idempotent** — an IO that already exists (by name, under the campaign) is reused, never duplicated — so it is safe to run repeatedly. Two ways to schedule:

A) Cron one-shot (recommended) — each tick runs one pass and exits:

```
*/5 * * * * cd /path/to/Promo-Operations && promo-ops poll-cases --live >> logs/poll.log 2>&
```

B) Long-running watcher — one process loops on an interval:

```
promo-ops poll-cases --live --watch --interval 300          # every 5 min, forever
promo-ops poll-cases --watch --interval 300 --max-cycles 10 # bounded (dry-run) test
```

For B under systemd, run it as a `simple` service with `Restart=on-failure`.

Each cycle logs a one-line summary (`cycle N: X processed (Y submitted, Z needs-info)`) plus per-Case results. A transient Salesforce error is caught and the loop continues on the next tick. Start with dry-run (omit `--live`) to watch it pick up test Cases.

Run log (audit trail). Add `--log-file logs/poll-runs.jsonl` to persist one JSONL record per cycle (timestamp, counts, per-Case case_id/IO link/needs-info/error). Review it any time with:

```
promo-ops poll-status --log-file logs/poll-runs.jsonl
```

which prints cumulative cycles/submitted/needs-info/errors and the last cycle's Cases.

Daily digest. For a shareable end-of-day summary (email/Slack), run:

```
promo-ops daily-digest --log-file logs/poll-runs.jsonl          # today
promo-ops daily-digest --log-file logs/poll-runs.jsonl --day 2026-08-01
```

It de-dupes Cases across cycles (latest state wins) and lists the drafts created (with IO links + placement counts) and any needs-info Cases with their reasons.

What's already proven (no creds needed)

- `build_plan_dict` (Case fields + Targeting → plan) — `tests/test_from_case.py` .
- `check_case_schema` (describe → missing fields/values) — `tests/test_salesforce_preflight.py` .
- The generated field spec stays in sync with `CASE_FIELD_MAP` — `tests/test_from_case.py` .